Module 03

K8s – K8s Pod Service ReplicaSet

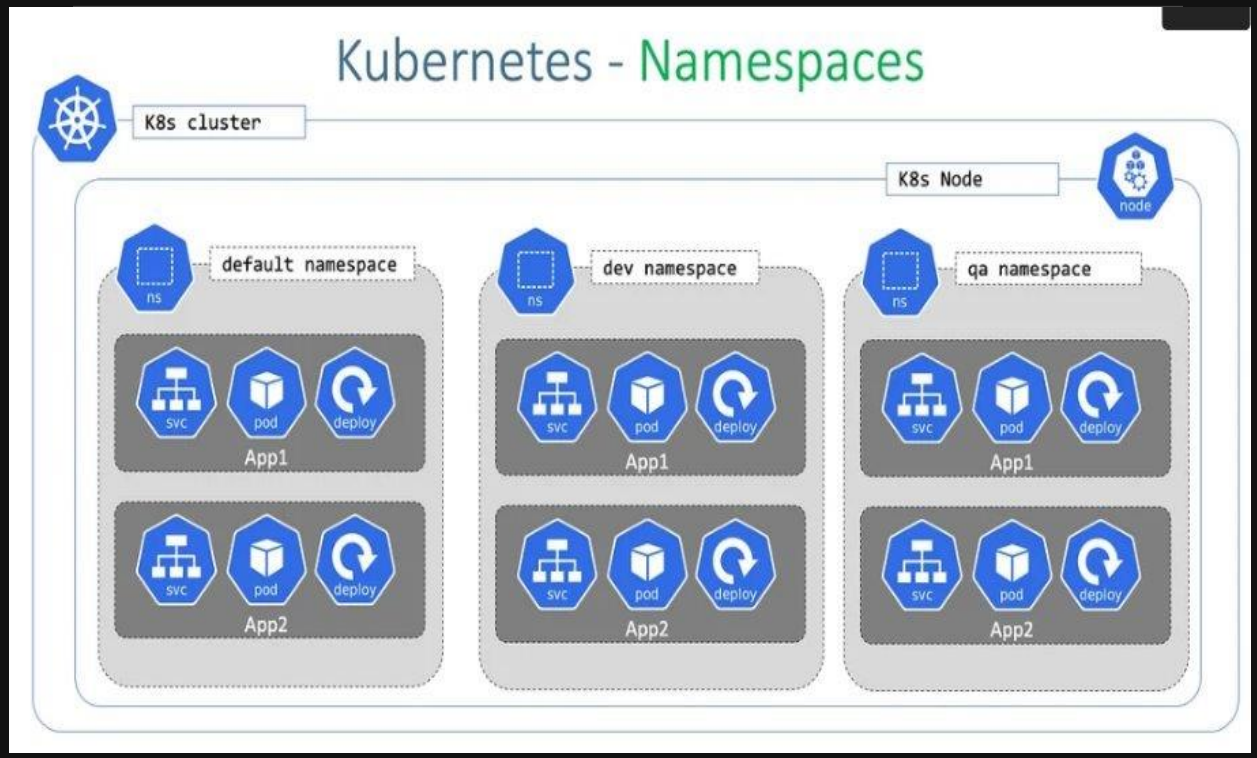
DevOps end to end services & solutions

Agenda

- Namespace
- Pod,Service and ReplicaSet
- Type of Services
- Hands-on

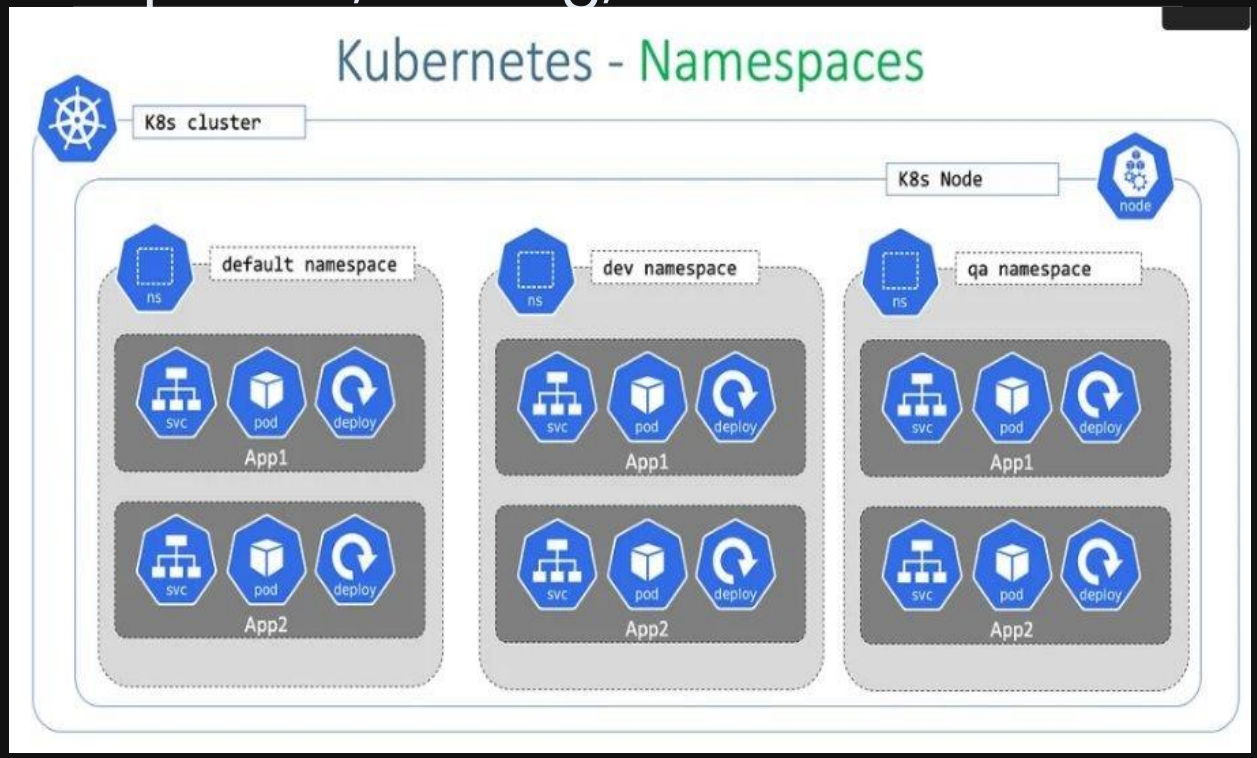
K8s – What is Namespace (ns) ?

- Namespaces: is a Resource Isolation which provide a way to divide cluster resources between multiple users, teams, or projects
- Resources in one ns are logically isolated from another ns
- The access control enable via Role-Based Access Control (RBAC) for fine-grained permissions on resources
- Resources are created in the default ns, in case there is no set a specified ns



Namespace – Common Use Cases

- Multi-Tenant Environments: Isolate workloads for different teams or applicatio in a shared cluster
- Environment Separation: Separate development, testing, and production environments.
- Resource Quota Management: Prevent one team or application from consuming all cluster resources
- Resource quotas can be applied to namespaces to limit resource consumption (e.g., CPU, memory).



Namespace – Built-In Namespaces

- Default: Used for resources with no explicitly defined namespace.

- Kube-system: Contains resources used by Kubernetes itself (e.g., CoreDNS, kube-proxy).

- Kube-public: A readable ns for cluster-wide public information (not comm only used).
  - kubectl apply -f namespace.yaml
  - kubectl create namespace <ns-name> without yaml file
  - kubectl get namespaces
  - kubectl get all -n <ns-name>
  - kubectl delete namespace <ns-name>
  - kubectl config set-context --current --namespace=<n-name>

yaml

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: example-namespace
```

K8s – What is POD ?

- A pod is the smallest execution unit in Kubernetes
- A pod encapsulates one or more applications
- Pods are ephemeral by nature, if a pod (or the node it executes on) fails, K8s can automatically create a new replica of that pod to continue operation
- A pod typically runs a single container, but it can host multiple tightly coupled containers that share resources
- Containers in the same pod share the same network namespace and can communicate via localhost

POD can Shared Resources

- Storage: Pods can access to shared volumes mounted to multiple containers.
- Networking: Each pod has a unique IP address within the cluster
- Pods are ephemeral, If a Pod fails, controllers like Deployments or ReplicaSets create a new replacement Pod.
- A pod runs an application, microservice, or process, either as a single container or as a group of tightly integrated containers
- Single-Container Pod, Running a standalone application or service
- Multi-Container Pod, Containers sharing resources

POD – Common Commands

- kubectl apply -f pod.yaml
- kubectl apply -f <pod-name.yaml>
- kubectl get pods
- kubectl get pods
- kubectl describe pod <pod-name>
- kubectl logs <pod-name>
- kubectl delete pod <pod-name>

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
  labels:
    app: my-app
spec:
  containers:
    - name: my-container
      image: nginx:latest
      ports:
        - containerPort: 80
```

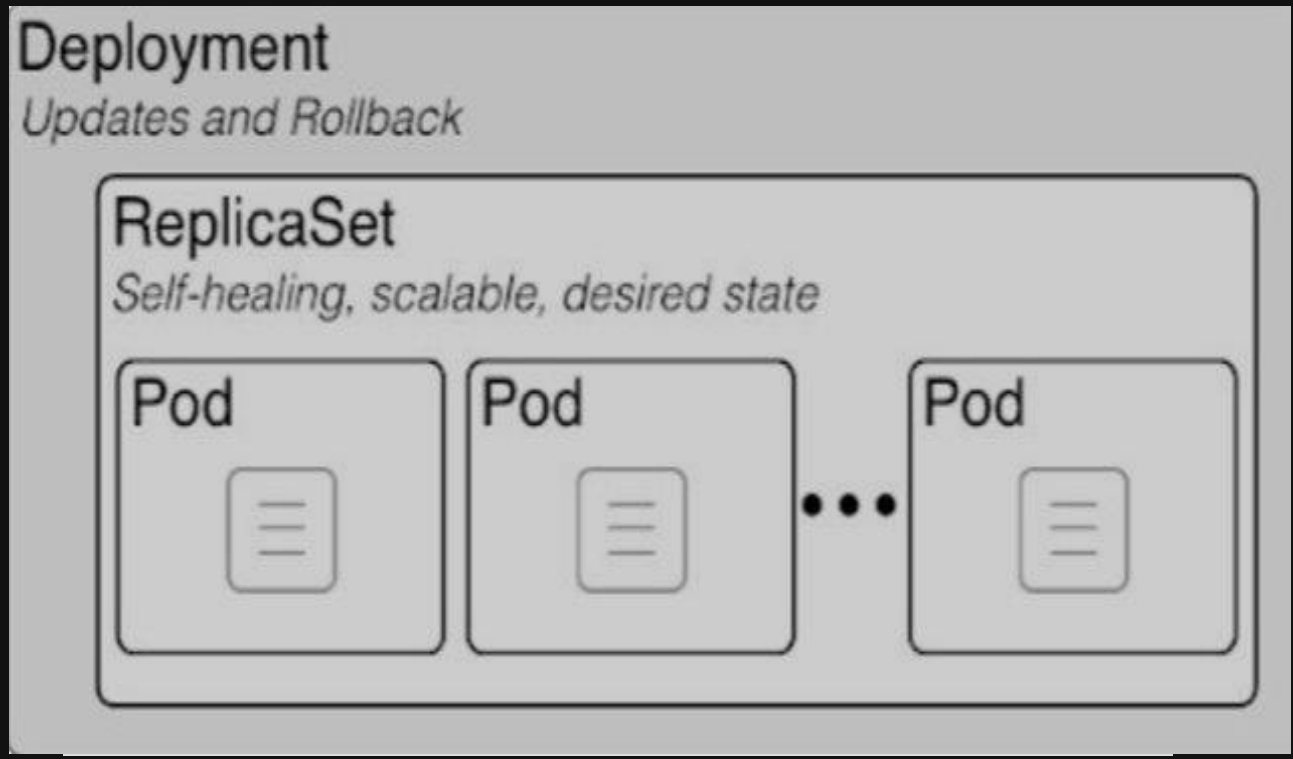
POD – Common Commands

- kubectl apply -f pod.yaml
- kubectl apply -f <pod-name.yaml>
- kubectl get pods
- kubectl get pods
- kubectl describe pod <pod-name>
- kubectl logs <pod-name>
- kubectl delete pod <pod-name>

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
  labels:
    app: my-app
spec:
  containers:
    - name: my-container
      image: nginx:latest
      ports:
        - containerPort: 80
```

K8s – What is ReplicaSet (rs) ?

- ReplicaSet is a resource that ensures a specified number of identical pod replicas are running at all times
- It is commonly used to maintain the desired state of an application
  By creating or deleting pods
  as necessary to match the specified
  Replica count



K8s – What is ReplicaSet (rs) ?

- Replication: Ensures the desired number of pod replicas are running atany given time
- Self-healing: Automatically replaces failed or deleted pods
- Label Selector: Uses labels to identify and manage a group of pods
  - Labels are key-value pairs attached to pods
  - Selectors determine which pods are managed by the rs based on their labels.
- Scalability: Allows you to scale applications up or down by adjusting the replica count.

K8s – ReplicaSet (rs) Yaml File

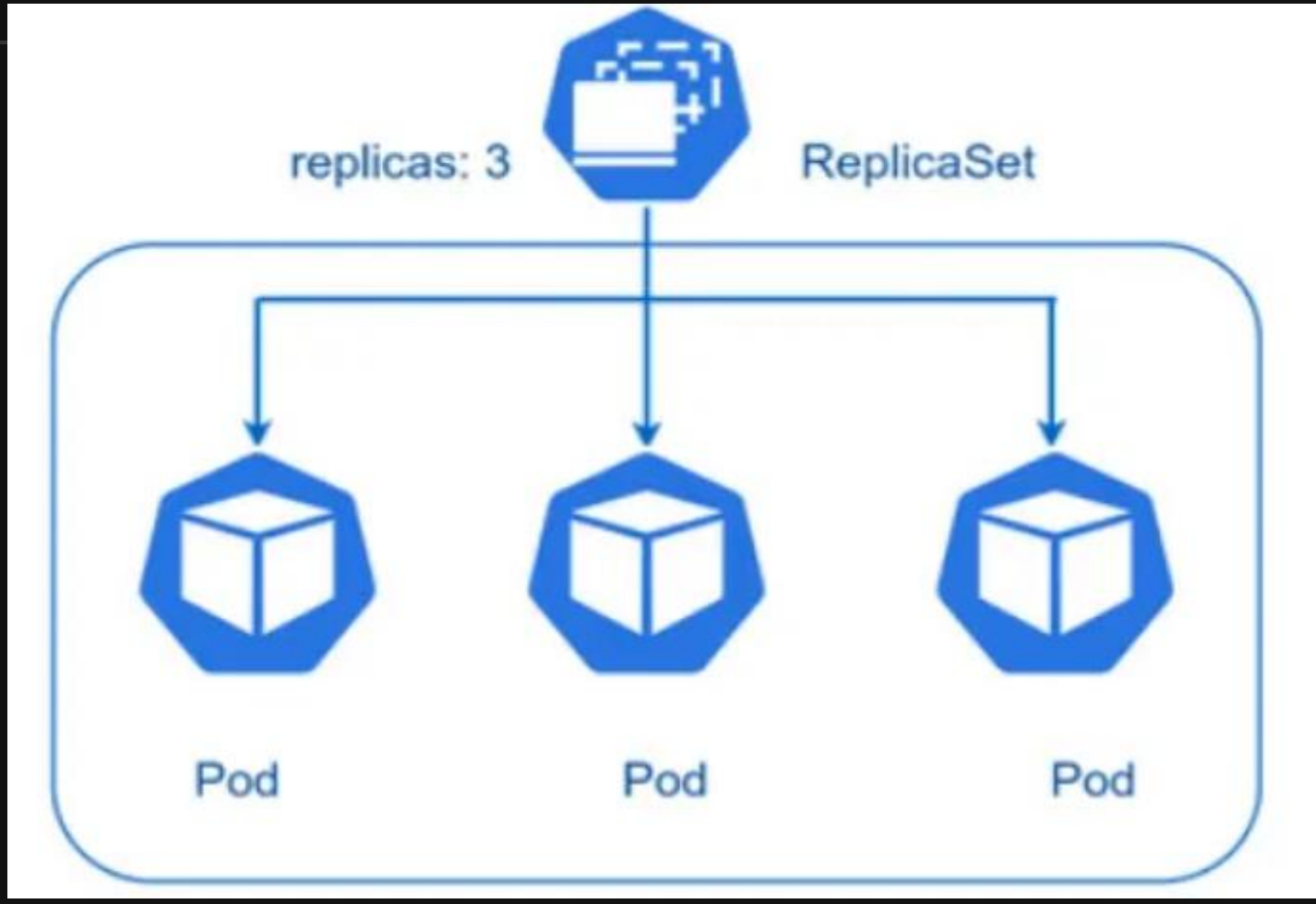
Explanation:

- ➢ replicas: Ensures 3 pods are running
- ➢ selector: Manages pods
  - with the label app: my-app
- ➢ template: Defines the pod specification
  - (e.g., container, image, and ports).

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: example-replicaset
  labels:
    app: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-container
          image: nginx:latest
          ports:
            - containerPort: 80
```

RS– Common Commands

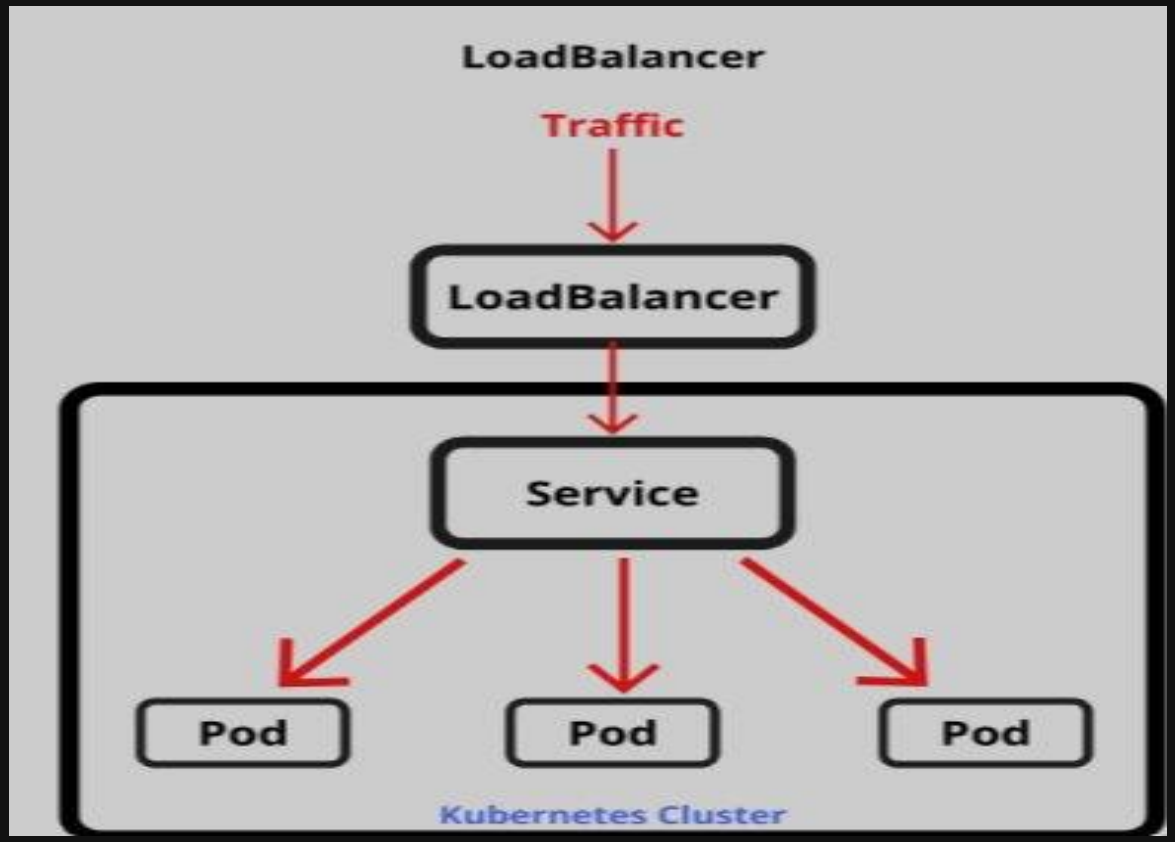
- kubectl apply -f replicaset.yaml
- kubectl get replicaset
- kubectl describe replicaset /
  - my-replicaset
- kubectl scale replicaset /
  - my-replicaset --replicas=5
- kubectl delete replicaset /
  - my-replicaset



K8s – What is Services (svc) ?

- A key aim of Services in K8s is that you don't need to modify your existing application to use an unfamiliar service discovery mechanism

  You can run code in Pods
- Whether this is a code designed for a cloud-native world or an older app you've containerized
- You use a Service to make that set of Pods available on the network so that clients can interact with it



K8s – What is Services (svc) ?

- A Service is a method for exposing a network application that is running as one or more Pods in your cluster.
- A Service is to group a set of Pod endpoints into a single resource
- By default, you get a stable cluster IP address that clients inside the cluster can use tocontact Pods in the Service
- Expose an application running in your cluster behind a single outward-facing endpoint, even when the workload is split across multiple backends

K8s – What is Services (svc) ?

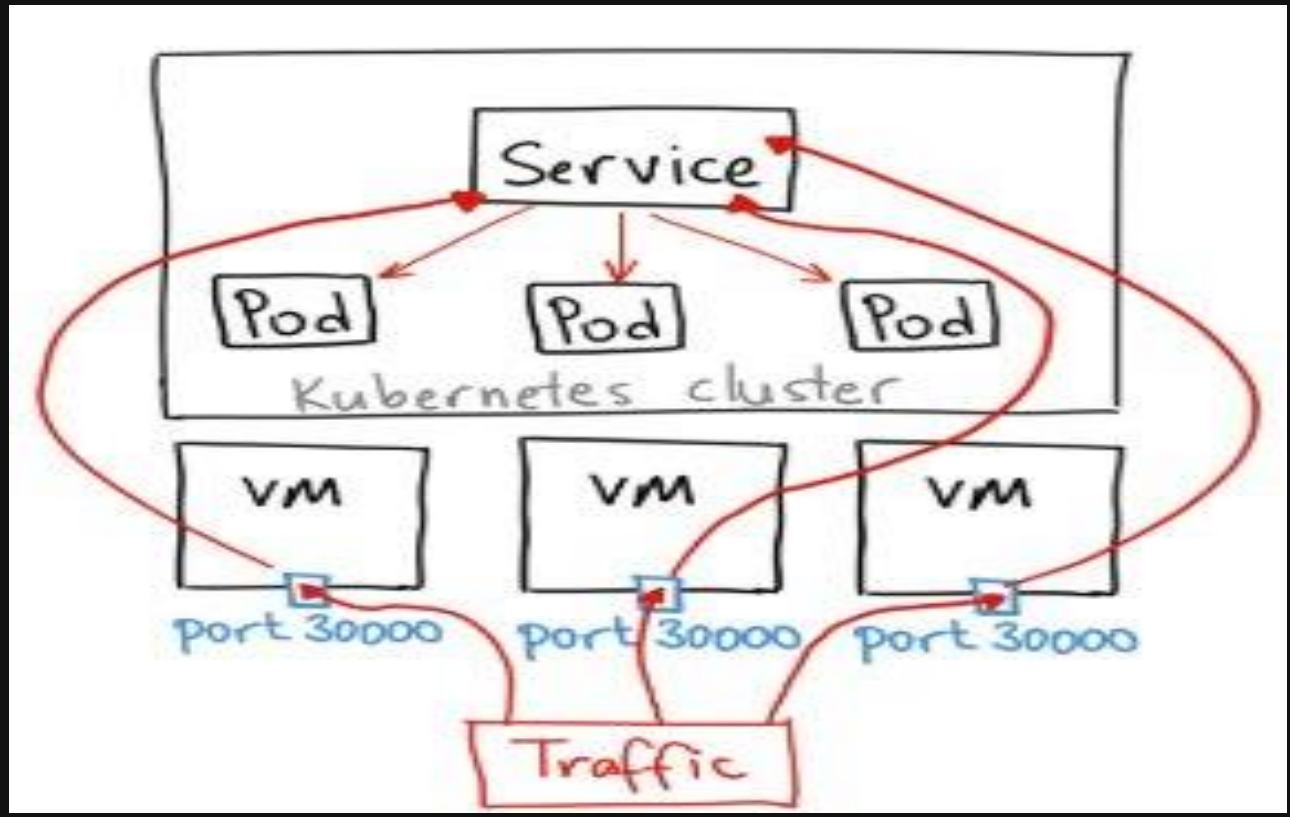
- A Service in Kubernetes is an abstraction that defines a logical set of Pods and provides a stable network endpoint for accessing them

- Services enable communication between different parts of an application or between external clients and the application, even as Pods are dynamically created, deleted, or moved.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: example-service
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
```

K8s – Why we need the Services (svc) ?

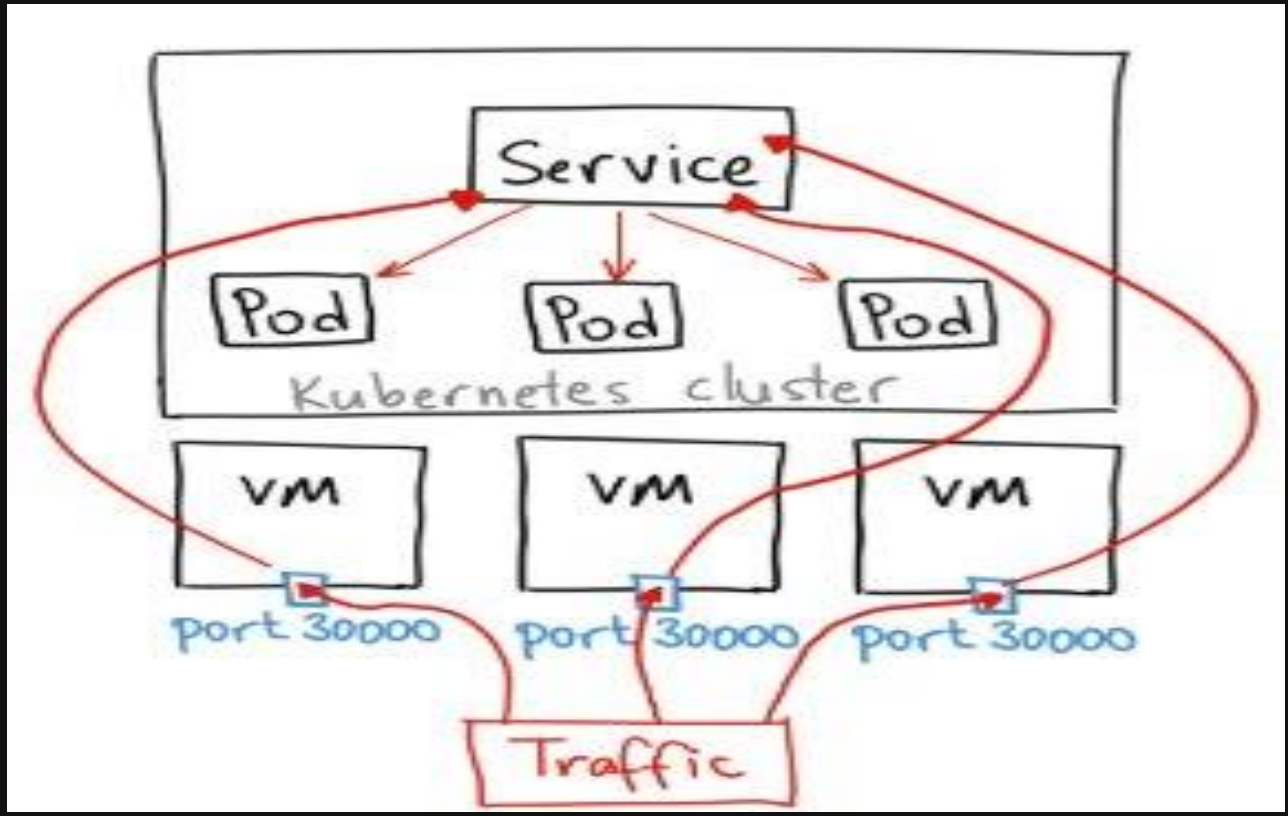
- Stable Networking: Provides a consistent IP address and DNS name to access a group of Pods, even as individual Pods are replaced or rescheduled
- Load Balancing: Distributes network traffic across multiple Pods, ensuring high availability and reliability



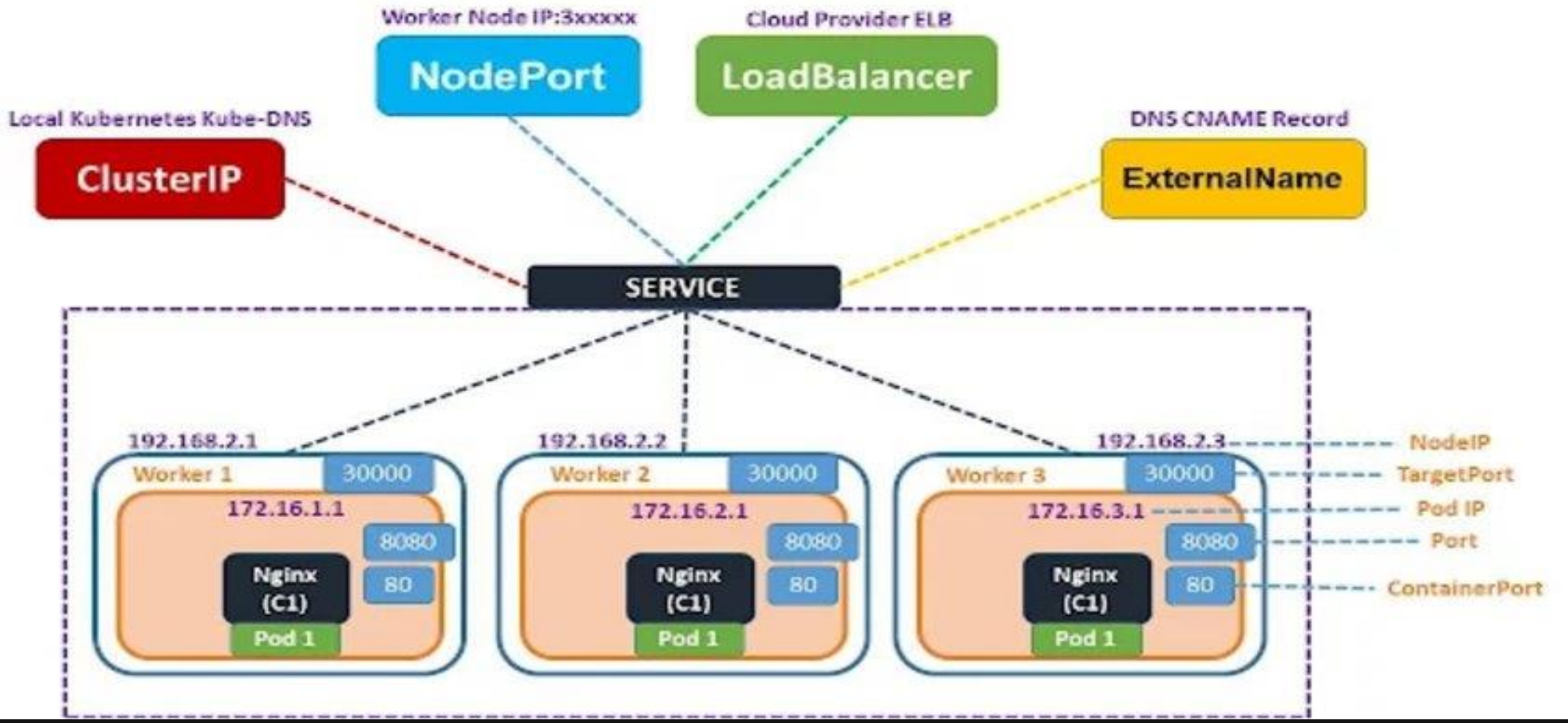
K8s – Why we need the Services (svc) ?

- Discovery: K8s assigns a DNS name to each Service, allowing applications to discover other services easily

- Selector-Based Targeting:
  Services use selectors to identify the Pods theyroute traffic to, based on labels Ports range used are between 30000-32767



K8s – Type of Services (svc) ?



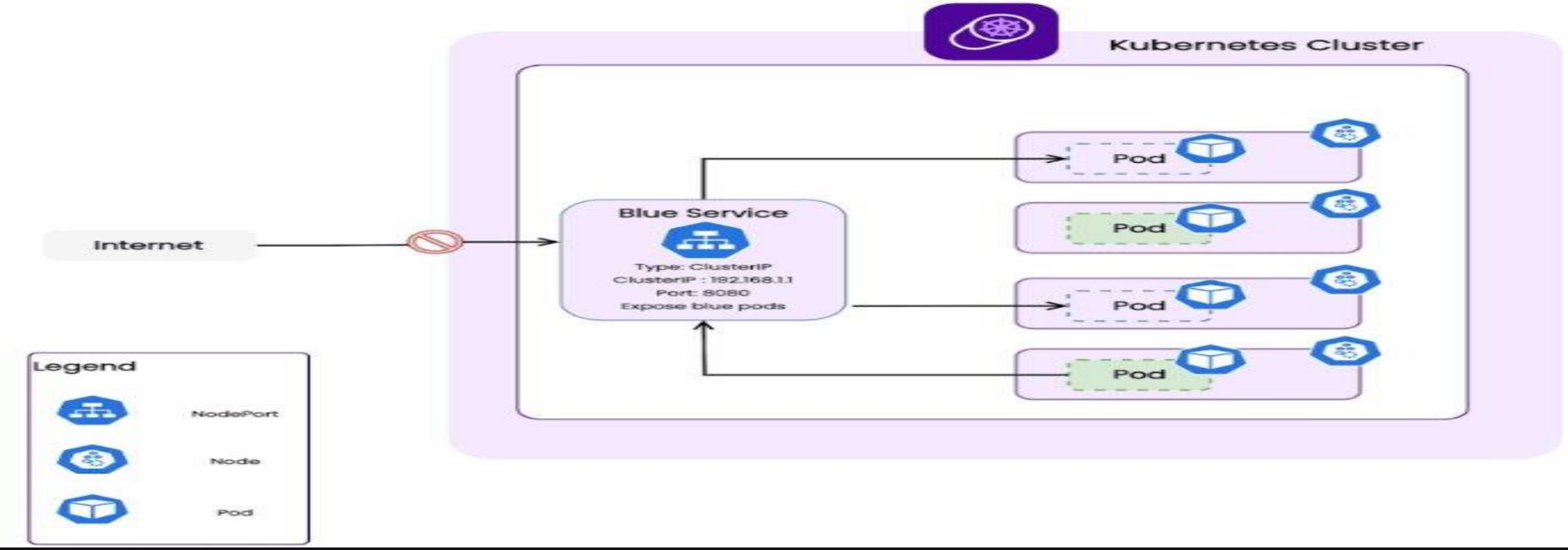
K8s – Type of Services (svc) Cluster IP

Suitable for internal communication between application components

Is the default K8s Service type

It exposes an application

- Inside the Kubernetes cluster only
- Not accessible directly from outside
- Stable virtual IP



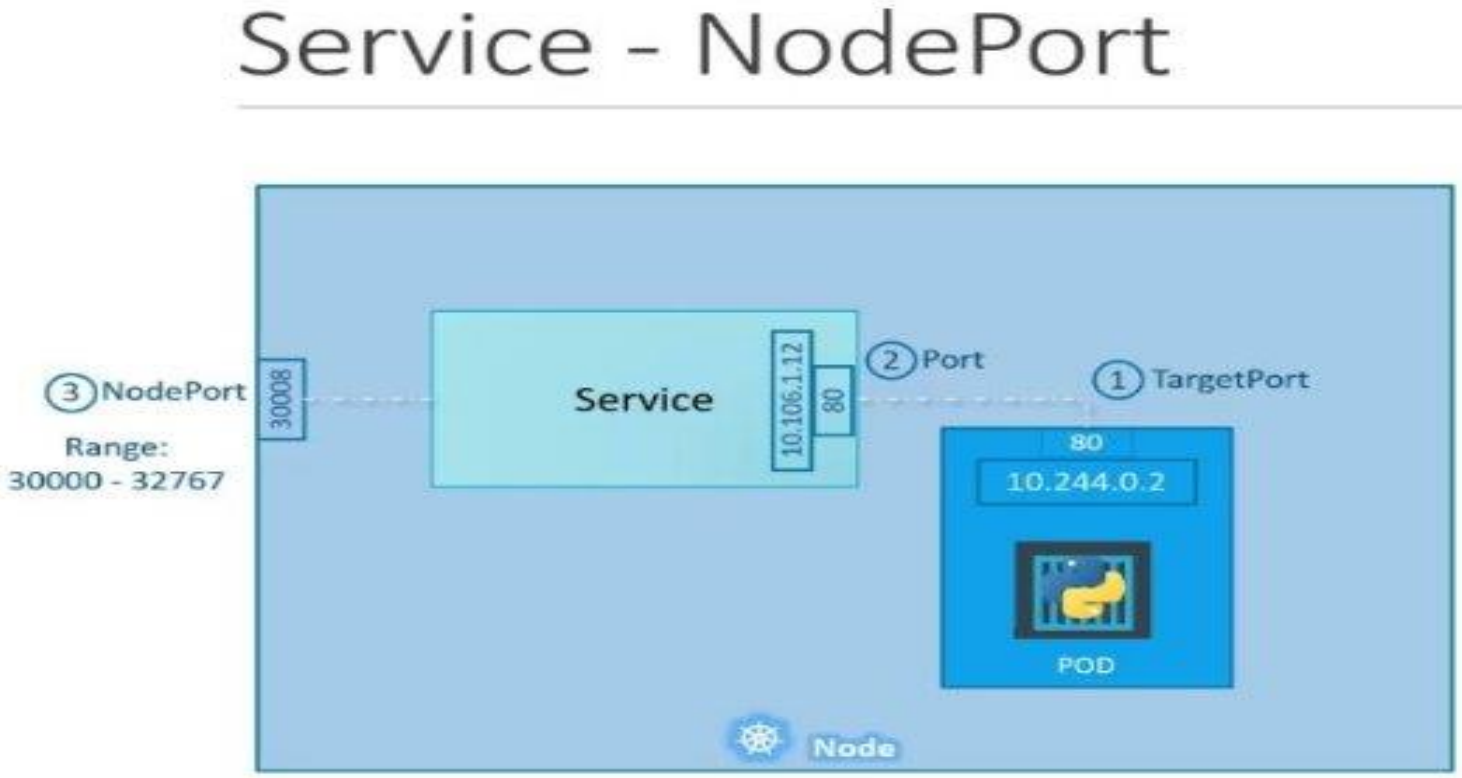
K8s – Type of Services (svc) NodePort

Exposes your application outside the cluster

using a port on every K8s node

Not Secured, Opens port on all nodes

- **Accessible from outside the cluster using <NodeIP>:<NodePort>.**
- **Useful for debugging or simple external access**



service-definition.yml

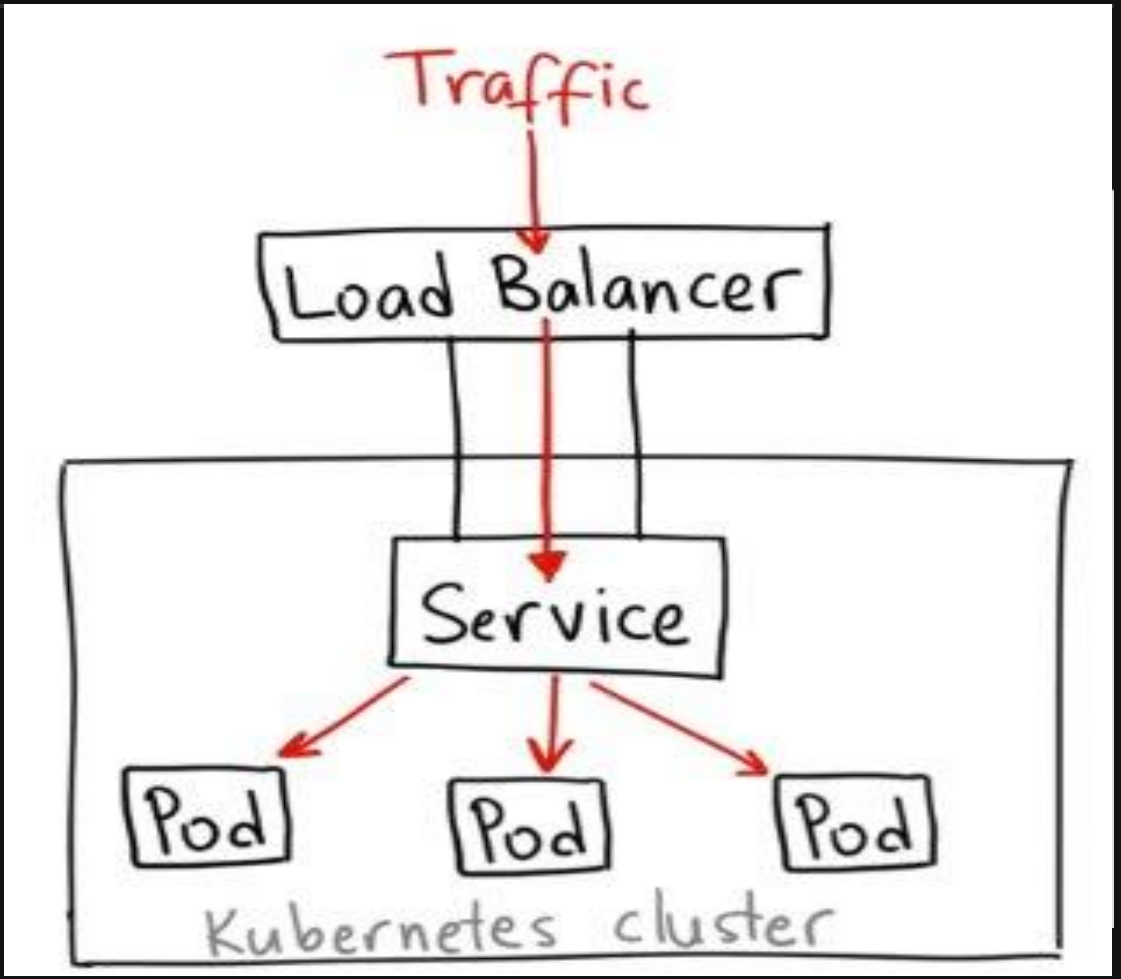
```yaml
apiVersion: v1
kind: Service
metadata:
    name: myapp-service

spec:
    type: NodePort
    ports:
      - targetPort: 80
        port: 80
        nodePort: 30008
```

K8s – Type of Services (svc) LoadBalancer

Is the easiest way to expose an app externally in cloud environments

- **Automatically provisions an external load balancer**
- **Routes traffic to the Service from outside the cluster**
- **K8s asks the cloud provider to create:**
  - **Public IP**
  - **External Load Balancer**
  - **Routing to Pods**

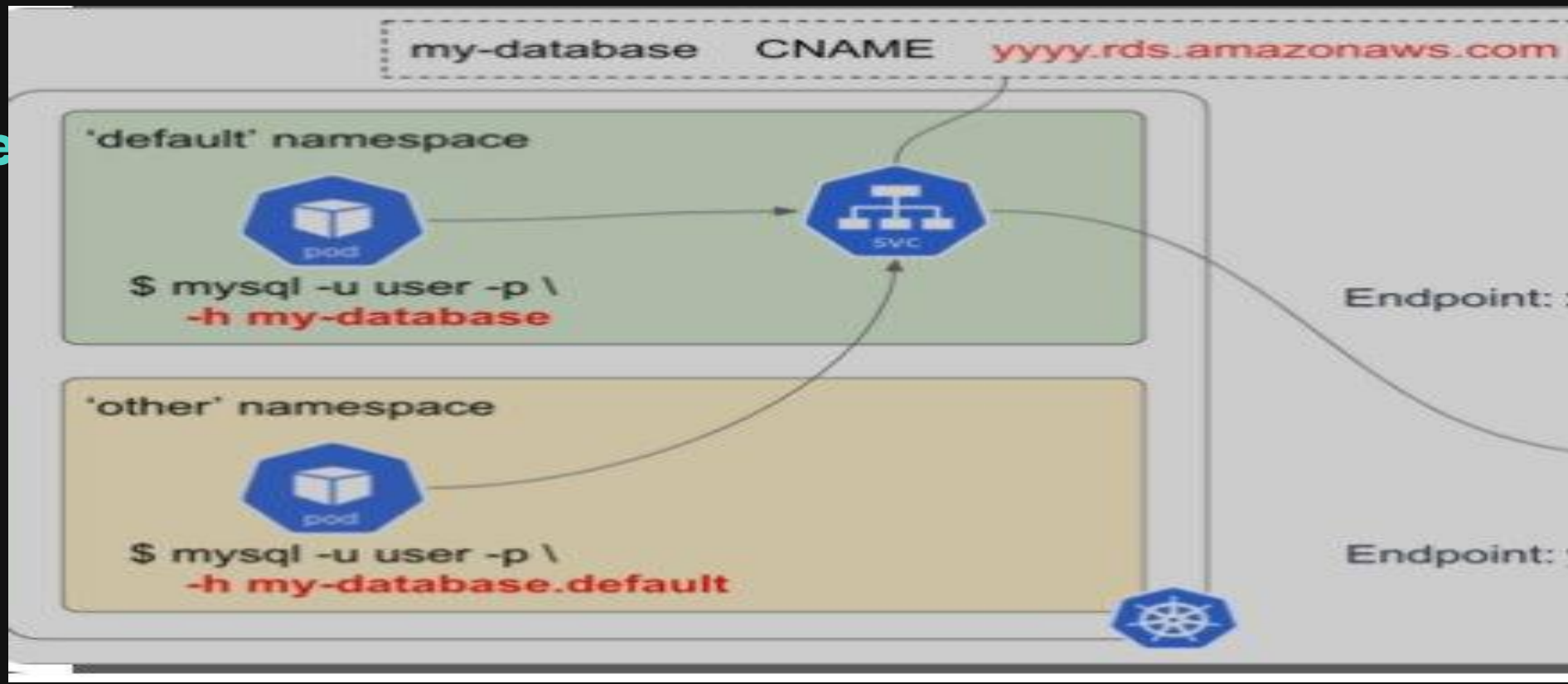


K8s – Type of Services (svc) ExternalName

- Maps the Service to an external DNS name
- Instead of forwarding traffic to Pods, it creates a DNS alias

Useful:

- External database
- External API
- Legacy system outside cluster SaaS services



K8s – Summary Type of svc

ClusterIP vs NodePort vs LoadBalancer vs ExternalName

Feature	ClusterIP	NodePort	LoadBalancer	ExternalName
Internal Access	✅ Yes	✅ Yes	✅ Yes	❌ No
External Access	❌ No	✅ Yes (`<NodeIP>:<NodePort>`)	✅ Yes (`<External-IP>:<Port>`)	✅ Yes (`DNS name`)
Automatic IP Assignment	✅ Yes	✅ Yes	✅ Yes	❌ No
Cloud Provider Required?	❌ No	❌ No	✅ Yes	❌ No
Use Case	Internal microservices	External access (testing)	Production cloud services	Connecting to external APIs

K8s – Kubectl common CLI commands

- kubectl cluster-info
- kubectl get pods --all-namespaces
- kubectl describe pod nginx-676b6c5bbc-h2p6m
- kubectl logs nginx-676b6c5bbc-h2p6m
- kubectl logs nginx-676b6c5bbc-h2p6m -c nginx
- kubectl exec -it nginx-676b6c5bbc-h2p6m --/bin/bash
- kubectl get svc
- kubectl describe svc nginx
- kubectl get namespaces
- kubectl config set-context --current --namespace=<>

Hands-On – Lab 04

Getting Started – Go To exercise 04

- devops-k8s
- /lesson-k8s
- /exercises
- /exercise-04
- /README.md

Questions?

Thank you for participating.

Feel free to reach out:
https://dinghy-e2e.com